



南開大學  
Nankai University

计算机学院

深度学习及应用（高阶课）实验报告

## 循环神经网络算法

姓名：申健强

学号：2313119

专业：计算机科学与技术

2026 年 6 月 24 日

# 目录

|     |                          |   |
|-----|--------------------------|---|
| 1   | 实验目的与要求                  | 2 |
| 2   | 数据集与实验设置                 | 2 |
| 3   | 模型结构与原理                  | 2 |
| 3.1 | RNN（老师原始版本，基线） . . . . . | 2 |
| 3.2 | LSTM（手写实现，加分项） . . . . . | 3 |
| 4   | 实验结果                     | 3 |
| 5   | 为什么 LSTM 优于 RNN（重点）      | 5 |
| 6   | 扩展：基于字符级 RNN 的名字生成       | 7 |
| 7   | 结论                       | 8 |

## 1 实验目的与要求

本实验的目标是掌握循环神经网络 (RNN) 的基本原理, 并使用 PyTorch 在名字识别任务 (根据一个姓名预测其所属的语言/国家, 共 18 类) 上完成以下工作:

- 复现老师提供的原始版本 RNN, 给出网络结构、在验证集上的训练 loss 曲线、准确率曲线以及预测 (混淆) 矩阵图;
- 自行实现 LSTM 网络完成同样的任务, 给出其结构、loss 曲线、准确率曲线与混淆矩阵图;
- 重点分析为什么 LSTM 的性能优于 RNN;
- 加分项: 自己实现 LSTM 单元 (不直接调用 `nn.LSTM`)。本报告的 LSTM 即为手写门控实现。

作为扩展, 我们另外实现了基于字符级 RNN 的名字生成任务, 单列一节讨论 (见第 6 节)。

## 2 数据集与实验设置

**数据集.** 采用 PyTorch 官方提供的 names 数据集, 包含 18 种语言、共 20,074 个姓名, 每种语言一个文本文件。每个名字先经 Unicode 转 ASCII 规范化, 再表示为字符 one-hot 序列: 字符表为大小写字母加 " . , ; ' " 与一个 OOV 占位符, 共 `n_letters=58` 个, 于是一个长度为  $L$  的名字被编码为形状  $(L, 1, 58)$  的张量。按 85%/15% 随机划分为训练集 (17,063) 与验证集 (3,011), 随机种子固定为 2024。

**训练协议.** 两个模型使用完全相同的设置以保证可比: 损失函数为负对数似然 `NLLLoss` (配合输出端 `LogSoftmax`), 优化器为 SGD (学习率 0.15)。训练沿用老师 notebook 的写法——按“伪 batch” (每 64 个样本) 累加单样本损失后再统一反向、做梯度裁剪 (阈值 3) 并更新一次参数; 隐藏层维度 128, 训练 15 个 epoch。我们在每个 epoch 结束后于验证集上评估一次 Top-1 准确率并记录, 用于绘制准确率曲线。代码结构上, 两个模型定义在 `models.py`, 数据/训练/评估工具在 `train_utils.py`, 可视化在 notebook 中完成。

## 3 模型结构与原理

### 3.1 RNN (老师原始版本, 基线)

基线使用 PyTorch 内置的 `nn.RNN` 逐字符读入名字, 取最后时刻的隐藏态经线性层与 `LogSoftmax` 得到 18 类的对数概率。普通 RNN 的隐藏态按下式递推:

$$\mathbf{h}_t = \tanh(W_{ih} \mathbf{x}_t + W_{hh} \mathbf{h}_{t-1} + \mathbf{b}). \quad (1)$$

其结构实现如下:

```
1 class NameRNN(nn.Module):                # baseline, uses nn.RNN
2     def __init__(self, input_size, hidden_size, output_size):
3         super().__init__()
4         self.rnn = nn.RNN(input_size, hidden_size)
5         self.h2o = nn.Linear(hidden_size, output_size)
6         self.softmax = nn.LogSoftmax(dim=1)
7     def forward(self, line_tensor):        # (seq_len, 1, input_size)
8         _, hidden = self.rnn(line_tensor)
```

```
9         return self.softmax(self.h2o(hidden[0]))
```

### 3.2 LSTM (手写实现, 加分项)

为完成加分项, LSTM 单元的四个门完全自己实现, 不调用 `nn.LSTM`。在时刻  $t$ , 给定输入  $\mathbf{x}_t$ 、上一隐藏态  $\mathbf{h}_{t-1}$  与上一细胞态  $\mathbf{c}_{t-1}$ :

$$\begin{aligned} \mathbf{i}_t &= \sigma(W_i[\mathbf{x}_t, \mathbf{h}_{t-1}]), & \mathbf{f}_t &= \sigma(W_f[\mathbf{x}_t, \mathbf{h}_{t-1}]), \\ \mathbf{g}_t &= \tanh(W_g[\mathbf{x}_t, \mathbf{h}_{t-1}]), & \mathbf{o}_t &= \sigma(W_o[\mathbf{x}_t, \mathbf{h}_{t-1}]), \\ \mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{g}_t, & \mathbf{h}_t &= \mathbf{o}_t \odot \tanh(\mathbf{c}_t). \end{aligned} \quad (2)$$

其中  $\mathbf{i}, \mathbf{f}, \mathbf{o}$  分别为输入门、遗忘门、输出门,  $\mathbf{c}_t$  为细胞态。四个门的线性变换合并为一个 Linear 后切分实现:

```
1 class NameLSTM(nn.Module):                # hand-written LSTM cell (bonus)
2     def __init__(self, input_size, hidden_size, output_size):
3         super().__init__()
4         self.hidden_size = hidden_size
5         self.x2h = nn.Linear(input_size + hidden_size, 4 * hidden_size)
6         self.h2o = nn.Linear(hidden_size, output_size)
7         self.softmax = nn.LogSoftmax(dim=1)
8     def forward(self, line_tensor):
9         h = torch.zeros(1, self.hidden_size)
10        c = torch.zeros(1, self.hidden_size)
11        for t in range(line_tensor.size(0)):
12            x = line_tensor[t]                # (1, input_size)
13            i, f, g, o = self.x2h(torch.cat([x, h], 1)).chunk(4, 1)
14            i, f, g, o = i.sigmoid(), f.sigmoid(), g.tanh(), o.sigmoid()
15            c = f * c + i * g                # cell state
16            h = o * torch.tanh(c)            # hidden state
17        return self.softmax(self.h2o(h))
```

## 4 实验结果

两个模型在相同协议下各训练 15 个 epoch, 总体结果见表 1。手写 LSTM 取得 **81.20%** 的验证准确率, 明显高于 RNN 的 78.74%。

表 1: RNN 与 LSTM 总体结果对比

| 模型            | 参数量    | 最佳准确率 (%)    | 最终准确率 (%)    | 训练耗时 (min) |
|---------------|--------|--------------|--------------|------------|
| RNN (基线)      | 26,386 | 78.74        | 78.45        | 9.2        |
| LSTM (手写, 加分) | 98,066 | <b>81.20</b> | <b>81.20</b> | 18.5       |

图 4.1 与图 4.2 分别为两个模型的训练 loss 曲线与验证准确率曲线 (同坐标系便于对比)。可见 LSTM 的 loss 下降更充分、验证准确率曲线整体在 RNN 之上, 且末段更平稳。

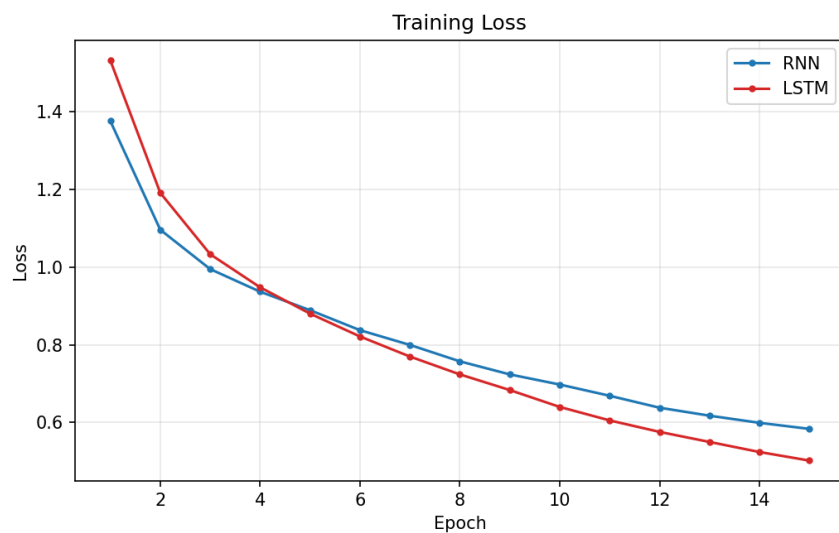


图 4.1: 训练 loss 曲线 (RNN vs LSTM)

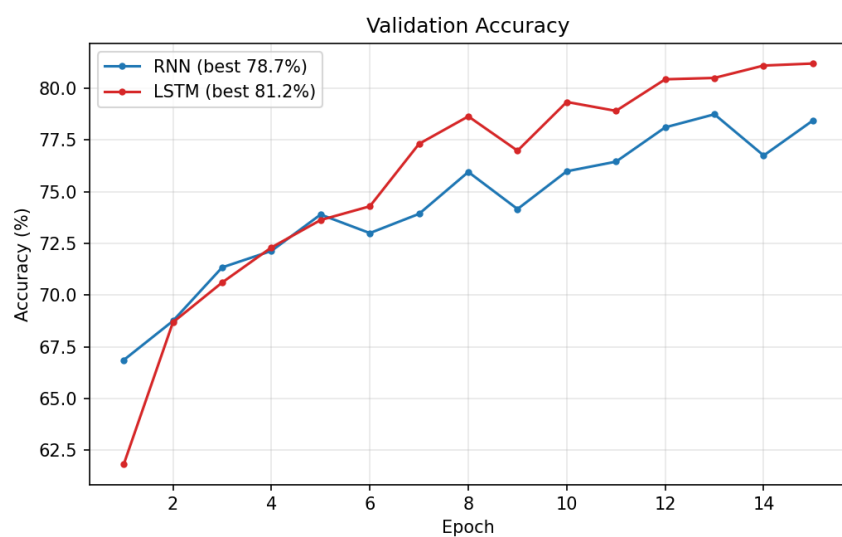


图 4.2: 验证准确率曲线 (RNN vs LSTM)

图 4.3 为两个模型在验证集上的预测（混淆）矩阵（按真实类别行归一化，对角线越亮表示该类识别越准）。

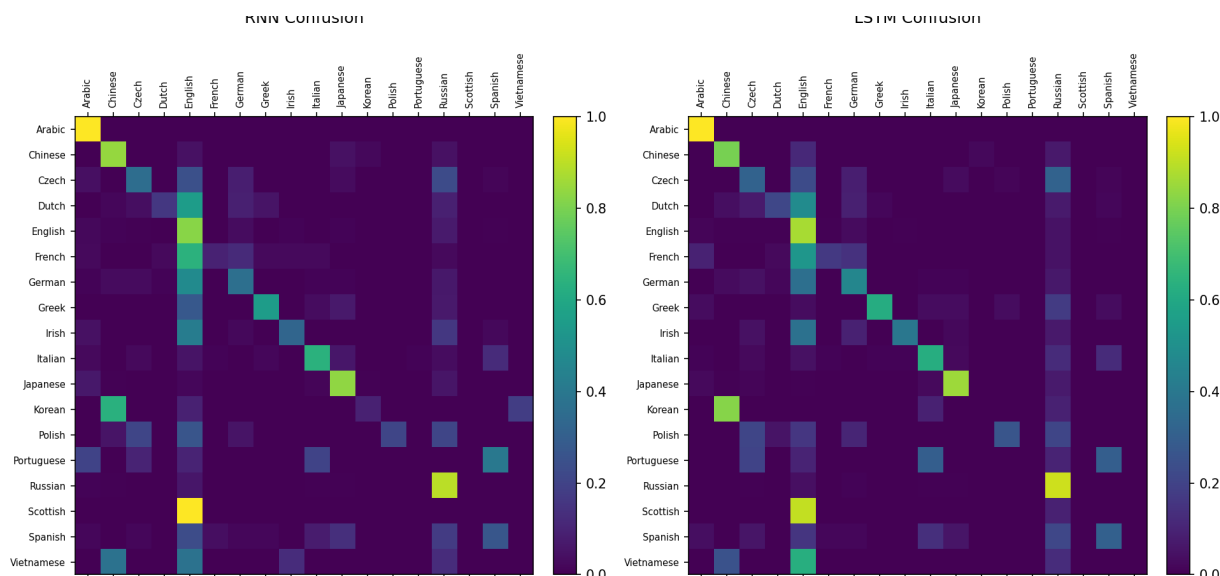


图 4.3: 预测 (混淆) 矩阵: 左 RNN, 右 LSTM

**逐类别观察.** 从混淆矩阵的对角线 (每类召回率) 看, LSTM 在多数类别上优于 RNN, 提升最明显的有 German (+9.2)、French (+7.1)、Irish (+7.0)、Greek (+6.9)、Dutch (+5.4)、Polish (+5.3)、English (+4.7) 等; 仅在 Chinese、Czech、Korean 等少数类别略有下降。此外, 数据集类别极不平衡 (如 Russian、English、Arabic、Japanese 样本众多, 而 Portuguese、Scottish、Vietnamese 等样本很少), 导致这些小菜两个模型的召回率都接近 0, 模型倾向于把它们误判为字符风格相近的大类——这也是“整体准确率”高于“各类别平均”的原因。

## 5 为什么 LSTM 优于 RNN (重点)

普通 RNN 仅靠单一隐藏态  $h_t$  在时间维上反复经非线性与矩阵相乘传播信息。在反向传播时, 梯度需连乘多个雅可比矩阵, 序列略长便容易出现**梯度消失或爆炸**, 使网络难以学习较早字符与类别之间的依赖; 对名字识别而言, 词尾、词中的关键字符信息在长名字里难以稳定保留。

LSTM 通过引入**细胞态  $c_t$**  和**输入门/遗忘门/输出门**解决了这一问题:

- 细胞态以  $c_t = f_t \odot c_{t-1} + i_t \odot g_t$  的**加性**方式更新, 当遗忘门  $f_t \approx 1$  时梯度可近似线性地沿  $c_t$  传播, 从而**缓解梯度消失**, 使长程依赖得以保留;
- 三个门是数据相关的开关: 输入门控制写入多少新信息, 遗忘门控制保留多少历史, 输出门控制暴露多少细胞态, 赋予网络**选择性记忆**的能力。

**机制与行为的双重验证.** 为把上述论述**可视化**而非停留在叙述, 我们从两个角度给出证据 (图 5.4)。其一 (左图, 梯度沿时间反传): 在**随机初始化**下度量输出对各时间步隐藏态的梯度  $\|\partial h_T / \partial h_t\|$ , 普通 RNN 随回溯距离**指数衰减** (回溯 18 步后降到约  $10^{-4}$ ); 而 LSTM 只有在**遗忘门打开** ( $f \approx 1$ ) 时细胞态才成为“**梯度高速路**”、衰减最慢 (比同距离的 RNN 高约 70 倍), 若遗忘门接近 0.5 则衰减反而比 RNN 更快——这定量地印证了“加性细胞态在  $f \approx 1$  时缓解梯度消失”这一论断, 也解释了为何遗忘门偏置常初始化为 1。其二 (右图, 真实任务行为): 随名字长度增加, LSTM 相对 RNN 的准确率优势从短名字的约 +1.9 个百分点扩大到 11 字符以上的约 +4.3 个百分点, 说明优势确来自对**更长上下文**的更好保持。需诚实指出: 本任务名字较短 (多为 5 ~ 10 字符), 梯度消失并非 LSTM 取胜的唯一原因, 门控带来的更强表达能力同样重要。

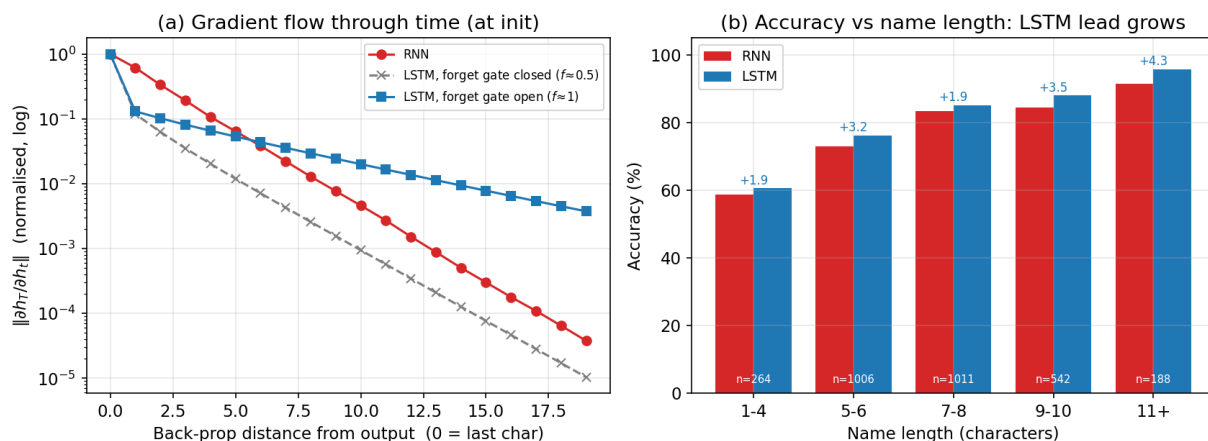


图 5.4: 为什么 LSTM 优于 RNN。左: 初始化时梯度沿时间反传的幅值  $\|\partial h_T / \partial h_t\|$  (对数纵轴) —— RNN 指数消失, LSTM 仅在遗忘门打开时保持梯度 (遗忘门关闭时甚至更差)。右: 真实任务上准确率随名字长度的变化, LSTM 的领先幅度随长度增大 (+1.9 → +4.3 pp)。

**逐字符的“读懂过程”**：图 5.5 把名字**逐字符**喂入分类器，展示预测语言的概率如何随每个字符演化（行 = 候选语言，列 = 已读字符，绿框 = 真实语言）。可见 LSTM 往往能在读完整个名字后**果断地“锁定”正确语言**（真类一行末尾显著变亮），而 RNN 易在中途被局部字符组合带偏（如读到“Majew”时一度高概率判成 Japanese），最终把握也更弱。这与“门控带来的选择性记忆使其更稳地整合整段序列信息”一致。

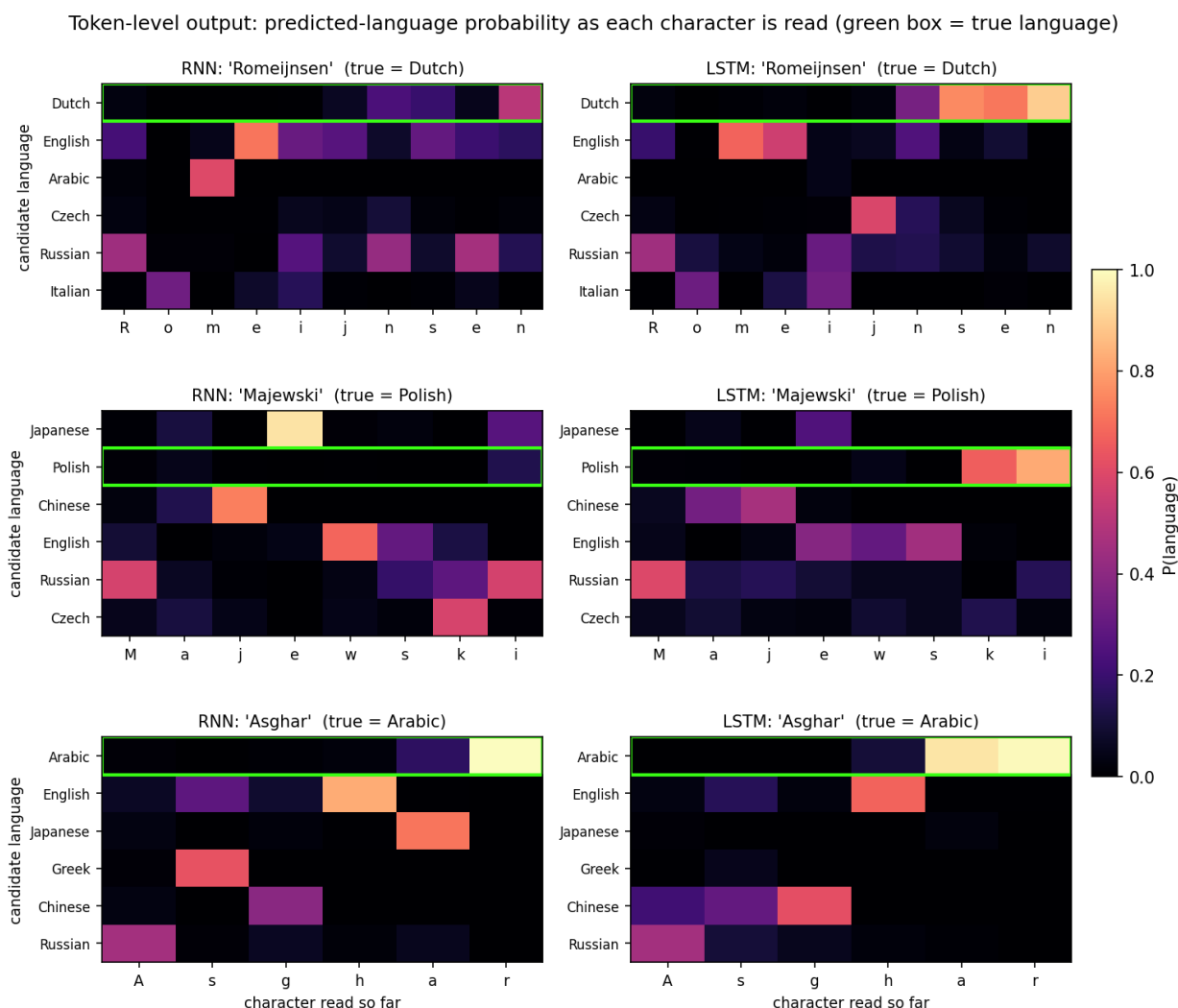


图 5.5: Token 级输出：逐字符读入时预测语言的概率演化（绿框 = 真实语言）。LSTM 末端对真类更果断，RNN 中途易被带偏。

这与实验结果一致：LSTM 的验证准确率（81.20%）高于 RNN（78.74%），收敛更稳，且在需要更长上下文才能区分的语言（如 German、French、Irish、Greek 等）上召回率提升显著（见上节逐类别观察）。代价是 LSTM 参数量（约 98K）与训练时间（18.5 min）大于 RNN（26K，9.2 min），属于以更高计算开销换取更强的序列建模能力。

## 6 扩展：基于字符级 RNN 的名字生成

除分类外，我们实现了名字生成任务：给定类别（语言）和起始字母，让网络逐字符地预测下一个字符，直到生成结束符 EOS。与分类只用最后一步输出不同，生成在**每一步**都做预测并累加损失。网络在每个时间步的输入为“类别 one-hot + 当前字符 one-hot + 上一隐藏态”的拼接，结构如下：

```

1 class RNN(nn.Module):                                # category-conditioned char generator
2     def __init__(self, input_size, hidden_size, output_size):
3         super().__init__()
4         self.hidden_size = hidden_size
5         self.i2h = nn.Linear(n_categories + input_size + hidden_size, hidden_size)
6         self.i2o = nn.Linear(n_categories + input_size + hidden_size, output_size)

```



```
7         self.o2o = nn.Linear(hidden_size + output_size, output_size)
8         self.dropout = nn.Dropout(0.1)
9         self.softmax = nn.LogSoftmax(dim=1)
10    def forward(self, category, input, hidden):
11        combined = torch.cat((category, input, hidden), 1)
12        hidden = self.i2h(combined)
13        output = self.o2o(torch.cat((hidden, self.i2o(combined)), 1))
14        return self.softmax(self.dropout(output)), hidden
```

训练时对每个名字，把相邻字符构成 (当前字符  $\rightarrow$  下一字符) 的监督对，按时间步累加 `NLLLoss` 后整体反向。

**采样策略的影响.** 最初采用**贪心解码** (每步取概率最大的字符)，存在两个问题：一是对同一 (类别, 起始字母) 输出完全确定、毫无多样性；二是容易卡在高频字符组合上重复，例如俄语名字会出现 `Uanovovov` 这类重复尾巴。改为**按概率分布采样并引入温度系数  $T$**  后多样性明显改善：

$$P'(\text{char}) \propto P(\text{char})^{1/T}, \quad (3)$$

$T$  越小越保守 (接近贪心)，越大越随机；名字生成取  $T \approx 0.5 \sim 0.8$  时既保持各语言风格又具有多样性。该实验说明：生成质量不仅取决于模型，也显著依赖解码/采样策略。

## 7 结论

本实验在名字识别任务上对比了 RNN 与手写实现的 LSTM。统一协议下，LSTM 以 81.20% 的验证准确率明显优于 RNN 的 78.74%，收敛更稳、在多数语言类别上召回率更高；其原因在于细胞态的加性更新与门控机制有效缓解了梯度消失、增强了长程依赖建模，代价是更大的参数量与训练时间。加分项方面，我们自行实现了 LSTM 的四门单元。扩展部分实现了字符级名字生成，并通过对比贪心与温度采样，说明了解码策略对生成多样性的重要影响。

## 参考文献

- [1] J. L. Elman. Finding Structure in Time. *Cognitive Science*, 14(2):179–211, 1990.
- [2] S. Hochreiter, J. Schmidhuber. Long Short-Term Memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [3] A. Paszke et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. *NeurIPS*, 2019.
- [4] S. Robertson. NLP From Scratch: Classifying / Generating Names with a Character-Level RNN. PyTorch Tutorials.